

Répartition des Tâches : Projet Mastermind

Ce document détaille la répartition des rôles et des tâches pour le développement du jeu Mastermind en Python, réalisé par l'équipe composée d'Arthur, Antony, Mohamed et Elouan, **incluant les mises à jour pour respecter le cahier des charges de IN200.**

1. Arthur : Intelligence Artificielle (IA)

Arthur a pris en charge le développement du mode où l'ordinateur doit deviner le code secret. Ses contributions se concentrent sur la prise de décision automatisée :

- **Création de l'IA basique** : Développement de la classe SimpleAI utilisée spécifiquement pour le mode 'IA devine'.
- **Génération de propositions** : Implémentation de la méthode next_guess() qui gère le premier essai de façon aléatoire, puis choisit une proposition parmi les possibilités restantes pour les tours suivants.
- **Filtrage par feedback** : Création de la méthode feedback(guess, noirs, blancs) qui simule le score de chaque code possible et élimine les combinaisons incompatibles avec les indices reçus.
- **Moteur d'Aide** : Réutilisation astucieuse de son IA en arrière-plan pour implémenter la fonctionnalité give_hint(), générant une proposition valide basée sur l'historique des coups du joueur.

2. Antony : Logique de validation (Guess) , sauvegarde et Gestion de version (GitHub)

Antony a géré le dépôt de code collaboratif et a implémenté le moteur mathématique et logique essentiel au bon fonctionnement du jeu :

- **Gestion du dépôt GitHub** : Mise en place de l'environnement de travail collaboratif, sécurisation du code et gestion de l'historique des versions.
- **Algorithme de calcul du score** : Conception de la fonction score(secret, guess) qui calcule le feedback. Cette fonction compte avec précision les pions ayant la bonne couleur à la bonne position (anciennement noirs, désormais rouges), et ceux de la bonne couleur à une mauvaise position (blancs), en tenant compte des fréquences des doublons.
- **Génération des combinaisons** : Développement de la méthode all_possible_codes() qui utilise itertools.product pour générer toutes les combinaisons possibles du jeu avec répétitions autorisées.
- **Logique d'Historique et Données** : Gestion des structures de données pour permettre l'annulation du dernier coup (undo_move()) et formatage de l'état pour les fonctions de sauvegarde JSON.

3. Mohamed : Interface Graphique (Affichage)

Mohamed a développé toute la partie visuelle (frontend) du jeu pour permettre aux joueurs d'interagir facilement. Ses réalisations :

- **Architecture de l'Application Tkinter** : Création de la classe principale MastermindApp et de la structure de base de la fenêtre.
- **Zones de jeu interactives** : Construction de l'interface avec la zone de code secret, l'historique des coups avec pions de feedback, et la zone de composition de la proposition.
- **Ajustement du Design (Nouveau)** : Modification du code visuel pour utiliser la couleur rouge au lieu du noir pour les pions bien placés, conformément aux règles du plateau de jeu.
- **Nouvelles Interactions** : Intégration des boîtes de dialogue natives (filedialog) et ajout des boutons graphiques pour les nouvelles fonctionnalités (Sauvegarder, Charger, Annuler le coup, Aide).

4. Elouan : Intégration, Configuration et Contrôle du Jeu (Propositions)

Puisque les autres membres se sont concentrés sur des modules bien spécifiques, Elouan a joué le rôle d'architecte, d'intégrateur et de testeur. Voici les tâches qu'il a assurées :

- **Intégration principale (Entry point)** : Écriture du fichier main.py pour instancier l'application Mastermind et lancer la boucle principale.
- **Configuration globale du jeu (Mise à jour)** : Définition des paramètres dans utils.py. Mise en conformité avec les règles officielles : passage à 8 couleurs disponibles (COLOR_MAX = 8, ajout du turquoise et bleu nuit) et réduction du nombre d'essais (MAX_TURNS = 10).
- **Logique de contrôle des modes de jeu** : Implémentation du système central de remise à zéro et de préparation pour orchestrer l'état selon le mode choisi.
- **Persistance des données** : Intégration du module json pour implémenter la sauvegarde (save_game) et le chargement (load_game), gérant la restauration complète de l'état du jeu et de l'interface.

Résumé des responsabilités

Membre de l'équipe	Responsabilité principale	Fichiers cibles impactés
Arthur	Développement de l'IA (choix, filtrage, fonctionnalité d'aide)	ai.py, ui.py
Antony	Algorithmes (Validation, Score), Gestion Undo & Git	utils.py, ui.py, GitHub
Mohamed	Interface utilisateur (Tkinter), design pions rouges, boutons	ui.py
Elouan	Intégration (Main), Configuration (8 couleurs, 10 essais), JSON	main.py, utils.py, ui.py